DESKTOP-DOM & AURA

The Definitive Technical Manual, Architectural Specification & Comprehensive Guide

From Core Fundamentals to Kernel Bridges & Autonomous Action Engines

---

Author & Engineer:	Piyush Dua (PDgit12)
Target Role:	Backend Developer Intern
Company / Founders:	Crcle.ai — Joshua Rayan & Cyril Rayan
Repository:	https://github.com/PDgit12/desktop-dom
Test Suite Health:	**83 / 83 Passing (100% Pass Rate)** in 3.2s
Packaging Releases:	macOS (.app, .dmg), Windows (.zip, .msi), Linux (.tar.gz), PyPI, npm
Active AI Engines:	Sub-25ms Zero-Model Fast-Path + Local Ollama (ministral-3:8b, qwen3:8b)

Executive Purpose: *This document serves as the exhaustive technical reference for* ***desktop-dom*** *and its companion floating assistant* ***Aura****. It is designed to take an engineer or founder from absolute first principles (what an operating system GUI tree is, how accessibility buses operate, why vision models fail on desktop automation) through every line of code, kernel adapter, mathematical algorithm, and design decision that powers sub-30ms, cursor-free, deterministic desktop automation.*

Chapter 1: Foundations & Prerequisites (From Absolute Zero)

1.1 What is an Operating System GUI?

A Graphical User Interface (GUI) is how humans interact with computers visually rather than through terminal commands. At the lowest hardware level, your graphics card and monitor display a two-dimensional grid of pixels (e.g., 3840×2160 on a 4K display). However, the operating system kernel (Darwin on macOS, Windows NT on Windows, Linux kernel on Linux) does not see applications as mere pictures. Behind every window running on your screen—whether it is Spotify, VS Code, Slack, or Chrome—is a running **process** (identified by a unique Process ID, or PID). That process manages user interface objects: windows, buttons, text fields, lists, and sliders.

1.2 What is a 'DOM' (Document Object Model)?

In web development, the term **DOM** (Document Object Model) refers to how a web browser represents a webpage in memory. When you open a website, the browser parses the raw HTML text (like <button>Click Me</button>) into a hierarchical tree of C++ objects in RAM. JavaScript tools like Playwright or Cypress can query this tree instantly: page.click('button#submit'). They do not guess where the button is visually; they inspect the tree and trigger the event directly.

The Desktop Analogy: For decades, desktop applications have lacked a unified DOM. macOS apps are written in Swift/AppKit, Windows apps in C#/WinUI, and cross-platform apps in Electron/Chromium. **desktop-dom** constructs a unified, cross-platform Semantic DOM for the entire desktop operating system, transforming native desktop applications into programmable, searchable tree structures just like web pages.

1.3 What is an Accessibility API (a11y) and Why Does It Exist?

Operating systems do not provide an explicit 'DOM' API for programmers. However, they do provide an **Accessibility API**. Thirty years ago, governments worldwide (e.g., Section 508 in the US, EN 301 549 in Europe) mandated that computers must be usable by people with disabilities. For a blind person to use a computer, a screen reader (such as *VoiceOver* on macOS, *NVDA* or *JAWS* on Windows, or *Orca* on Linux) must be able to inspect what is on the screen, read the text aloud, and activate buttons via keyboard commands.

To make screen readers work, Apple, Microsoft, and the Linux Foundation built deep operating system buses: **ApplicationServices AXUIElement** on macOS, **Microsoft UIAutomation (UIA)** on Windows, and **AT-SPI2 D-Bus** on Linux. Every compliant desktop app is required to expose its UI elements, names, roles, coordinates, and action triggers to this accessibility bus. **The Key Insight:** Screen readers needed the exact same structural data that AI agents need today! By tapping into native accessibility APIs, desktop-dom gets pixel-accurate bounding boxes, element roles, titles, and action triggers directly from the OS kernel without taking a single screenshot.

1.4 What is an AI Agent and a ReAct Planning Loop?

A standard Language Model (like GPT-4 or Claude) is just a text-in, text-out predictor. An **AI Agent** connects a model to a continuous execution loop. The standard architecture is called **ReAct** (Reasoning + Acting):

- **Thought:** The model analyzes the user request ('Play Starboy on Spotify') and the current state of the screen.
- **Action:** The model chooses a tool to call via structured JSON (e.g., click('btn_search_3a7f')).
- **Observation:** The tool executes on the OS and returns the newly mutated state to the model.
- **Loop:** The agent repeats this loop until the goal is verified as complete.

1.5 What is Local AI (Ollama) & Quantization?

Traditionally, developers call remote cloud APIs (OpenAI, Anthropic) using API keys. This introduces privacy risks, cost, and latency. **Ollama** is a local inference server that runs open-weight models (like Mistral, Llama 3.2, Qwen 2.5) directly on your laptop's CPU or Apple Silicon GPU. **Quantization** (e.g., Q4_K_M or int8) compresses 16-bit neural network weights down to 4 or 8 bits, reducing memory usage from 16GB to ~4GB with virtually zero loss in reasoning capability. Aura communicates with local Ollama on http://localhost:11434 for 100% offline, zero-cloud-cost reasoning.

1.6 What is Digital Audio & Speech Processing?

Microphones capture continuous sound waves. The computer samples these waves thousands of times per second (e.g., 16,000 times per second, or 16 kHz) as raw numbers called **Pulse Code Modulation (PCM)**. Aura captures audio into an in-memory NumPy array. To avoid burning CPU by running Whisper speech recognition on dead silence, Aura computes the **Root Mean Square (RMS) energy** of each 100ms chunk. If the room is silent (RMS < 0.015), the audio is instantly discarded with zero CPU overhead (<0.5% idle CPU). When voice energy is detected, faster-whisper (using CTranslate2) converts the audio into text in under 250 milliseconds.

Chapter 2: The Core Problem — Vision Agents vs. DOM Agents

In 2024–2026, the technology industry witnessed the rise of 'Computer Use' models (Claude 3.7 Computer Use, Gemini 2.0 Operator, Adept). All of these systems rely on a **vision-first pipeline**: taking a screenshot, passing the image to a massive vision-language model, predicting (x, y) pixel coordinates, and synthesizing hardware mouse clicks. While conceptually simple, this architecture suffers from 5 fundamental flaws:

Metric / Dimension	Vision-Based Agents (Claude / Gemini)	Desktop-DOM (Semantic Accessibility)
Payload Size per Step	1.5MB – 8MB uncompressed image	**1KB – 4KB structured JSON**
Token Consumption	1,500 – 2,000 multimodal tokens ($0.03/step)	**180 – 350 text tokens (85% reduction)**
Latency per Action	3,000ms – 5,500ms (screenshot + inference)	**15ms – 80ms Fast-Path; 600ms Local LLM**
Coordinate Precision	Drifts on Retina 2× scaling & multi-monitors	**Exact floating-point centroids from OS kernel**
User Cursor Interruption	**Hijacks physical cursor**; steals user focus	**Cursor-Free execution**; user continues typing
Hardware Cost & Privacy	Requires cloud API keys; exposes private screen	**100% Local**; 0MB RAM Fast-Path or local Ollama
Semantic State Awareness	Guesses whether buttons are disabled/checked	**Exact OS flags** (enabled, focused, selected)

The Conclusion: *Vision models are operating at the wrong layer of abstraction. For native desktop software, the operating system already possesses a structured, semantic, coordinate-accurate description of every element on screen. Desktop-DOM exposes this underlying reality directly to AI models.*

Chapter 3: System Architecture & Data Schema

The desktop-dom codebase is structured into five strictly decoupled architectural layers:

Layer 5: User Interfaces	**Aura Floating Omnibar** (Cocoa NSPanel + WebKit HUD), **Typer CLI** (desktop-dom doctor, apps, inspect), and **Stdio MCP Server** for Claude Desktop and Cursor.
Layer 4: High-Level SDK	DesktopApp (attach, launch, click, type, press) with reactive mutation observers (wait_for, wait_until_hidden) and LangChain tool exports.
Layer 3: Assistant Engine	AssistantBrain (dual-engine router with <25ms Fast-Path and local Ollama ReAct loop) + AudioManager (Whisper STT, RMS gating, OS TTS).
Layer 2: Normalization & Pruning	DOMPruner ($O(N)$ zero-area/offscreen elimination, 96% reduction), deterministic ephemeral ID generation, and FuzzyResolver for stale reference healing.
Layer 1: Kernel Platform Adapters	Native OS bindings: MacOSAdapter (PyObjC AXUIElement/Quartz), WindowsAdapter (UIA COM/SendInput), and LinuxAdapter (AT-SPI2 D-Bus).

3.2 The Core Schema (`src/desktop_dom/schema.py`)

Every UI element across all three operating systems is normalized into a standard Pydantic v2 data model:

```
class DesktopNode(BaseModel): id: str # Deterministic ID: e.g. 'btn_play_3a7f' role: str # Normalized role: 'button', 'textfield', 'window' name: Optional[str] = None # Accessible label or title value: Optional[str] = None # Text input content or slider level bounds: BoundingBox # [x, y, width, height] in desktop coordinates state: ElementState # enabled, focused, selected, checked children: List[DesktopNode] = [] # Hierarchical child elements class BoundingBox(BaseModel): x: float; y: float; width: float; height: float @property def centroid(self) -> Point: return Point(x=self.x + self.width / 2, y=self.y + self.height / 2)
```

Chapter 4: OS Kernel Adapters & Native Bridges

4.1 macOS Kernel Adapter (`src/desktop_dom/adapters/macos.py`)

On macOS, desktop-dom interfaces directly with the **ApplicationServices C-framework** via PyObjC. Key technical mechanics:

- **AXUIElementCreateApplication(pid):** Obtains an accessibility proxy pointer to the target process.
- **Unpacking PyObjC Pointer Out-Parameters:** In C, AXUIElementCopyAttributeValue takes a pointer to receive the result. In Python, PyObjC handles this by passing None as the pointer arg, returning a tuple (err, value). Checking err == 0 verifies success.
- **Cocoa vs Quartz Coordinate Inversion:** Cocoa (NSScreen) defines coordinate (0,0) at the *bottom-left* of the screen with Y increasing upward. Quartz (Window Server and AXUIElement) defines (0,0) at the *top-left* with Y increasing downward. Desktop-DOM normalizes everything to Quartz space and applies the formula Cocoa_Y = Screen_Height - (Quartz_Y + Window_Height) when moving Cocoa windows.
- **Electron / Chromium Accessibility Hydration:** Chromium-based apps (Chrome, Slack, VS Code, Spotify) keep their accessibility trees disabled by default to save 20MB of memory. Desktop-DOM sets AXEnhancedUserInterface = True and AXManualAccessibility = True on the root process, triggering Chromium's internal BrowserAccessibilityManager to construct its accessibility tree in under 25ms.

4.2 Windows Kernel Adapter (`src/desktop_dom/adapters/windows.py`)

On Windows, desktop-dom uses Microsoft's modern **CUIAutomation8 COM interface**. Standard traversals suffer from massive IPC overhead: reading 5 properties across 500 nodes requires 2,500 cross-process COM calls (~450ms). Desktop-DOM creates an IUIAutomationCacheRequest specifying required attributes up front and walks the ControlViewWalker. The entire tree is prefetched in a single batched IPC roundtrip taking under 20ms. Physical input fallback uses Win32 SendInput with hardware scan codes.

4.3 Linux Kernel Adapter (`src/desktop_dom/adapters/linux.py`)

On Linux, accessibility runs over the session D-Bus daemon (org.a11y.Bus). Desktop-DOM traverses org.a11y.atspi.Accessible objects and invokes AtspiAction for cursor-free triggers. Zero-area subtrees are clipped immediately to prevent unnecessary D-Bus roundtrips. Input synthesis supports both X11 (XTest) and Wayland (ydotool).

Chapter 5: The Cursor-Free Architecture ('Don't Disturb My Cursor')

The single most critical usability bottleneck of existing AI agents is **Cursor Hijacking**. When an agent takes control of the physical mouse to click buttons on screen, the human user is locked out. If the user attempts to type an email or move their mouse while the agent runs, the agent clicks the wrong location. Desktop-DOM completely eliminates cursor hijacking using a **3-tier non-disruptive execution hierarchy**:

Tier 1: Direct OS Action Invocation *(Zero Cursor Movement, Zero Focus Theft)*	Instead of synthesizing mouse events, desktop-dom calls AXUIElementPerformAction(elem, kAXPressAction) on macOS or InvokePattern.Invoke() on Windows. This triggers the button's internal event handler directly inside the target app's event loop. The physical mouse pointer never moves, the target window does not steal keyboard focus, and the user can continue typing in their active window.
Tier 2: Ghost-Click Restoration *(Sub-millisecond Warp & Restore)*	If a non-standard custom element does not implement kAXPressAction and requires coordinate clicking, desktop-dom: 1. Captures current cursor coordinates via Quartz.CGEventGetLocation in nanoseconds. 2. Dispatches mouse-down and mouse-up events at the target centroid. 3. Instantly warps the cursor back to the user's position using Quartz.CGWarpMouseCursorPosition in <1ms. The user experiences zero mouse drift.
Tier 3: In-Memory Value Injection *(Direct Memory Text Write)*	For text inputs, instead of synthesizing conflicting keyboard strokes, desktop-dom sets AXUIElementSetAttributeValue(elem, kAXValueAttribute, text). The text appears instantly inside the target field in RAM, without stealing focus from the document the human user is actively writing.

Chapter 6: Algorithms — Normalization, Pruning, & Stale Recovery

6.1 The $O(N)$ DOM Pruner (`src/desktop_dom/pruner.py`)

A raw desktop window contains thousands of invisible or passive layout containers. Desktop-DOM's DOMPruner executes a single-pass $O(N)$ tree descent that eliminates 96% of irrelevant nodes:

1. **Zero-Area Pruning:** Discards any element where width ≤ 0 or height ≤ 0.
2. **Offscreen Clipping:** Discards elements scrolled outside the active window boundary.
3. **Passive Container Flattening:** An unlabeled layout group (like an AXGroup with no title) is removed, and its interactive children are hoisted directly to the parent node.

> ***Benchmark:*** *On a complex 1,093-node window, the pruner reduces the tree to* ***44 interactive semantic nodes in 0.31 milliseconds****.*

6.2 Deterministic Ephemeral ID Generation

AI agents require compact, human-readable identifiers. Desktop-DOM computes deterministic IDs using: ID = <role_prefix>_<slug(name)>_<hash4(parent_path + relative_index)>. For example, a play button becomes btn_play_3a7f. If two identical buttons exist (e.g. duplicate 'Close' buttons), sibling collision avoidance appends ordinal indexes (btn_close_1, btn_close_2).

6.3 The `FuzzyResolver` (Self-Healing Generational Recovery)

When an application mutates dynamically (e.g. infinite scrolling), cached IDs can become stale. Instead of crashing, the FuzzyResolver computes a composite confidence score across the active DOM:

```
$$\text{Score} = (0.50 \times \text{LevenshteinSimilarity}) + (0.30 \times \text{RoleMatch}) + (0.20 \times \text{CentroidProximity})$$
```

If the top candidate's confidence exceeds **0.78**, desktop-dom automatically recovers the new node, heals its internal cache, and proceeds without throwing an exception.

Chapter 7: Targeted Subregion Vision Fallback

What happens when an application renders custom pixels on an HTML5 <canvas>, WebGL, or DirectX viewport (such as Figma, Canva, Google Maps, or video games) where the accessibility bus has no inner child nodes?

Instead of falling back to a wasteful full-screen 4K capture, desktop-dom uses **Targeted Subregion Vision** (src/desktop_dom/subregion_vision.py):

1. Desktop-DOM locates the canvas element in the accessibility DOM (e.g. canvas_figma_viewport).
2. Reads its exact bounding box: [x: 240, y: 120, width: 900, height: 600].
3. Uses native OS screen capture (CGWindowListCreateImage on macOS, BitBlt on Windows) to crop **only that 900×600 pixel bounding box**.
4. Passes the cropped image to the multimodal model and projects predicted relative coordinates back to global desktop display space.

Architectural Advantage: *Saves 80% image bandwidth, prevents exposure of private user data in adjacent windows or menu bars, and eliminates retina coordinate scaling ambiguity.*

Chapter 8: The Aura Assistant Engine & Audio Architecture

8.1 The Dual-Engine Intelligence Router (`brain.py`)

Aura implements a dual-engine architecture: combining an ultra-fast deterministic Fast-Path with an on-device neural ReAct loop.

- **Sub-25ms Fast-Path (0MB RAM):** Common workflows (Spotify media playback, adjusting hardware volume, toggling dark mode, creating Apple Notes, inspecting clipboard, screen introspection) do not require neural token generation. They execute deterministically via regex/AST dispatch in <25ms with 0MB RAM overhead.
- **Safe AST Math Calculator (Zero Vulnerabilities):** Mathematical queries (e.g. 'calculate 125 * 40 + 15') are parsed using Python's ast module. Arbitrary code execution via eval() is strictly prohibited, and exponentiation (**) is blocked to prevent algorithmic complexity DoS attacks.
- **On-Device ReAct Planning Loop:** Complex queries inject the pruned semantic tree into a local Ollama model (e.g. ministral-3:8b, qwen3:8b) on http://localhost:11434, executing autonomous multi-step desktop workflows.

8.2 Frictionless 1-Click Model Switcher

Aura dynamically queries http://localhost:11434/api/tags on startup. Clicking the model tag in the Omnibar or typing /model slides open the Model Drawer, allowing 1-click switching between installed models (ministral-3:8b, qwen3:8b) and the Zero-Model Fast-Path.

8.3 Audio Pipeline: Zero-Disk Whisper & RMS Wake-Word Gating (`audio.py`)

Traditional voice assistants write temporary .wav files to disk, creating flash wear and I/O latency. Aura captures audio directly into an in-memory NumPy float32 circular buffer and feeds it directly into faster-whisper (CPU/int8). Continuous wake-word listening calculates the Root Mean Square (RMS) energy of 100ms frames. Silent frames are discarded instantly, maintaining idle CPU usage at **under 0.5%**.

Chapter 9: The Floating Spotlight Omnibar (`Cocoa` + `WebKit`)

Aura's user interface is a native macOS floating pill inspired by Raycast and Spotlight (src/desktop_dom/assistant/omnibar.py):

- **Native Cocoa NSPanel:** Created with style masks NSWindowStyleMaskBorderless and NSWindowStyleMaskNonactivatingPanel. It hovers at NSFloatingWindowLevel and sets canJoinAllSpaces = True to follow the user across full-screen spaces.
- **Hardware-Accelerated Frosted Vibrancy:** Backed by an NSVisualEffectView with material NSVisualEffectMaterialHUDWindow positioned underneath a transparent WKWebView, blending smoothly with macOS desktop wallpapers.
- **Multi-Display Mouse Tracking:** On summon (Cmd+Shift+Space), show() queries Cocoa.NSEvent.mouseLocation() to detect which monitor currently contains the user's cursor, centering the Omnibar on that specific screen.
- **Two-Way WebKit IPC Bridge:** JavaScript posts messages via window.webkit.messageHandlers.desktopDom.postMessage to OmnibarScriptHandlerObjC. Python dispatches results back to JavaScript on the main thread via NSOperationQueue.mainQueue().addOperationWithBlock_.
- **The Result Drawer Pattern:** When an action executes, the Omnibar does not blink away. It expands to show a formatted output card with an engine latency badge (■ Fast-Path • 18ms vs ■ Ollama • 840ms), an instant [■ Copy] button, and [Done (Esc)].

Chapter 10: Hermetic Testing & Cross-Platform Packaging

10.1 Hermetic Testing Architecture (100% Pass Rate Across 83 Tests)

A critical engineering requirement for production infrastructure is **hermetic CI**: tests must execute reliably in headless Linux containers without physical monitors, window servers, or hardware microphones.

Desktop-DOM achieves this via tests/conftest.py, which implements an in-memory mock calculator tree adapter. The test suite covers schema validation, pruner algorithms, fuzzy recovery, reactive timeouts, multi-display negative coordinate calibration, audio thread concurrency, and packaging scripts across **83 tests passing in 3.2 seconds**.

10.2 Cross-Platform Release Packaging Pipeline

The unified packaging script (scripts/build_app.py & desktop-dom package --platform all) builds native releases:

- **macOS:** Bundles Aura.app with portable source trees, renders AppIcon.icns using Pillow, and creates a 753 KB drag-and-drop .dmg installer via hdiutil.
- **Windows:** Generates multi-resolution aura.ico, a WiX Toolset XML specification (AuraInstaller.wxs) for building .msi installers, and a standalone .zip distribution.
- **Linux:** Compiles standard Debian package directory trees (DEBIAN/control, /usr/bin/aura) and release tarballs.
- **Libraries:** Python package validated with twine check and TypeScript SDK (@desktop-dom/core) verified with npm pack --dry-run.

Chapter 11: Founder Interview Playbook (Crcle.ai Defense)

11.1 The 30-Second Elevator Pitch

*"Vision-based computer agents like Claude and Gemini are burning 90% of their tokens on redundant 2MB screenshots, struggling with 3-second latency loops, and drifting across retina display scaling. We built **desktop-dom** as 'Playwright for Native Desktop Apps'—extracting semantic OS accessibility DOMs in sub-30ms, enabling cursor-free background execution without focus theft, and allowing local 8B models to drive desktop workflows with zero cloud compute cost."*

11.2 Core Founder Questions & Engineering Answers

Q: Why not fine-tune a vision model like Claude Computer Use?

A: Vision models operate at the wrong layer of abstraction. A 4K screenshot is 8 million raw pixels. Turning that into 2,000 vision tokens to click a button that already has an exact OS identifier introduces latency, high token costs, and coordinate drift. By querying native accessibility trees directly, we get exact roles, states, and coordinates in 15ms with 85% fewer tokens. We reserve vision strictly as a subregion fallback for canvas viewports where no accessibility nodes exist.

Q: How does desktop-dom solve the 'Cursor Hijack' problem?

A: We built a 3-tier execution hierarchy. In Tier 1, we call AXUIElementPerformAction(kAXPressAction) on macOS or InvokePattern on Windows. This triggers the button's internal event handler with zero physical cursor movement and zero window focus theft. In Tier 2, if coordinate clicks are required, we record the cursor position, click, and warp back in <1ms via CGWarpMouseCursorPosition. In Tier 3, we set text fields directly in memory (kAXValueAttribute) so the user can type in another window simultaneously.

Q: What happens when an app mutates and cached IDs break?

A: We implemented a generational FuzzyResolver. Element IDs are deterministic composite strings: role_slug_hash4. When an ID fails to resolve, FuzzyResolver computes a weighted similarity score using Levenshtein distance on element names, role validation, and spatial centroid proximity. If candidate confidence exceeds 0.78, the agent heals the reference and executes without interruption.

Q: How does this align with Crcle.ai's mission?

A: Crcle is building the intent layer that translates user goals into action across desktop software. The biggest barrier to agent adoption is reliability and speed: users won't tolerate 4-second delays or having their mouse stolen while they work. Desktop-DOM provides the deterministic, sub-30ms, cursor-free execution engine Crcle needs, with open MCP tool interfaces ready to plug into any orchestrator.